

Cẩm nang: 5 Quy trình tự động hóa Node.js Low-Code

5 automation thực tế có thể copy, chỉnh biến môi trường và chạy cho blog, dịch vụ nhỏ, dashboard hoặc quy trình nội bộ.

Công nghệ & AI / Automation

Dùng tài nguyên này như thế nào?

Chọn một quy trình lặp lại nhiều nhất trong công việc của bạn, copy khung script, thay biến môi trường và chạy trên dữ liệu mẫu trước khi chạy thật.

Bản thiết kế nội dung: HarryShare.vn

Cập nhật: 08/2026 - Free Resource

1. Low-code ở đây nghĩa là gì?

Low-code không có nghĩa là không cần hiểu gì về code. Trong tài liệu này, low-code nghĩa là bạn dùng các đoạn script nhỏ, cấu hình biến môi trường và kết nối API có sẵn để tự động hóa việc lặp lại. Bạn không cần xây cả phần mềm lớn, nhưng vẫn cần biết dữ liệu đi từ đâu đến đâu.

Nên dùng khi nào?

Dùng khi công việc lặp lại hàng tuần, dữ liệu có cấu trúc, lỗi do copy-paste nhiều, hoặc cần báo cáo định kỳ. Không nên dùng khi quy trình còn thay đổi mỗi ngày hoặc dữ liệu chưa có format ổn định.

Kiến trúc automation tối thiểu



2. Chuẩn bị chung

- Node.js 20+ và npm/pnpm.
- File .env để lưu token, database URL, spreadsheet ID. Không commit file này lên Git.
- Một nơi lưu dữ liệu: SQLite/PostgreSQL/Supabase/Google Sheet.
- Một cách chạy định kỳ: cron, GitHub Actions, Vercel Cron, hoặc task scheduler.
- Một kênh báo lỗi: email, Telegram, Slack, Discord hoặc log file.

3. 5 quy trình thực tế

Quy trình 1: Đồng bộ đơn hàng từ Google Sheet về DB nội bộ

Vấn đề giải quyết

Dữ liệu đơn hàng nhập trong Google Sheet nhưng website/dashboard cần đọc từ database để lọc, thống kê hoặc hiển thị.

Luồng xử lý



Các bước triển khai

1. Tạo Google Sheet có hàng tiêu đề rõ ràng: orderId, name, email, product, status.
2. Tạo service account hoặc OAuth để đọc sheet.
3. Dùng Prisma upsert để tránh trùng đơn hàng.
4. Ghi log số bản ghi tạo mới/cập nhật.
5. Chạy định kỳ mỗi 15-60 phút tùy nhu cầu.

Code mẫu / khung triển khai

```
const { google } = require('googleapis');
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function syncOrders() {
  const auth = new google.auth.GoogleAuth({
    scopes: ['https://www.googleapis.com/auth/spreadsheets.readonly'],
  });
  const sheets = google.sheets({ version: 'v4', auth });
  const res = await sheets.spreadsheets.values.get({
    spreadsheetId: process.env.SPREADSHEET_ID,
    range: 'DonHang!A2:E',
  });

  for (const row of res.data.values || []) {
    const [orderId, name, email, product, status] = row;
    if (!orderId) continue;
    await prisma.order.upsert({
      where: { id: orderId },
      update: { name, email, product, status },
      create: { id: orderId, name, email, product, status },
    });
  }
}

syncOrders().catch(console.error).finally(() => prisma.$disconnect());
```

Ưu và nhược điểm

Ưu điểm	Nhược điểm
Nhanh, ít lỗi nhập lại, phù hợp quy mô nhỏ.	Nếu Sheet bị đổi cột hoặc thiếu dữ liệu, script dễ lỗi. Cần validate input.

Quy trình 2: Form đăng ký tài nguyên -> gửi email/Telegram xác nhận

Vấn đề giải quyết

Người dùng tải tài nguyên nhưng bạn không biết ai đăng ký, nguồn nào chuyển đổi tốt, hoặc có lỗi form mà không phát hiện.

Luồng xử lý



Các bước triển khai

1. Tạo endpoint nhận webhook từ form.
2. Validate email và resourceId.
3. Lưu lead vào DB.
4. Gửi thông báo Telegram/email cho admin.
5. Trả về link tải hoặc gửi email xác nhận.

Code mẫu / khung triển khai

```
async function handleLead(req, res) {
  const { email, resourceId, source } = req.body;
  if (!email || !email.includes('@')) {
    return res.status(400).json({ error: 'Invalid email' });
  }

  await prisma.lead.create({ data: { email, resourceId, source } });
  await fetch(`https://api.telegram.org/bot${process.env.BOT_TOKEN}/sendMessage`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      chat_id: process.env.CHAT_ID,
      text: `Lead mới: ${email} tải ${resourceId} từ ${source || 'unknown'}`,
    }),
  });

  return res.json({ ok: true, downloadUrl: `/resources/${resourceId}.pdf` });
}
```

Ưu và nhược điểm

Ưu điểm	Nhược điểm
Biết tài nguyên nào có người tải, phản hồi nhanh khi form lỗi.	Cần xử lý consent/email policy nếu dùng để gửi email marketing.

Quy trình 3: Tự tạo metadata cho bài viết Markdown/MDX

Vấn đề giải quyết

Khi viết nhiều bài, dễ thiếu title, slug, description, category, date hoặc cấu trúc frontmatter không đồng nhất.

Luồng xử lý



Các bước triển khai

1. Quét thư mục content.
2. Tìm file thiếu frontmatter.
3. Tạo slug từ tiêu đề.
4. Gợi ý description ngắn.
5. Báo lỗi nếu thiếu category hợp lệ.

Code mẫu / khung triển khai

```
const fs = require('fs');
const path = require('path');

const VALID_CATEGORIES = ['tu-duy-san-pham', 'thuong-hieu-ca-nhan', 'cong-nghe-ai', 'hanh-trinh-lam-nghe'];

function slugify(input) {
  return input.toLowerCase()
    .normalize('NFD').replace(/[-]/g, '')
    .replace(/đ/g, 'd').replace(/^[^a-z0-9]+/g, '-')
    .replace(/^-|$/g, '');
}

function validatePost(filePath) {
  const raw = fs.readFileSync(filePath, 'utf8');
  const title = raw.match(/^#\s+(.+)/m)?.[1];
  if (!title) return { filePath, error: 'Missing H1 title' };
  return { filePath, slug: slugify(title), ok: true };
}

fs.readdirSync('content').filter(f => f.endsWith('.mdx'))
  .map(f => validatePost(path.join('content', f)))
  .forEach(result => console.log(result));
```

Ưu và nhược điểm

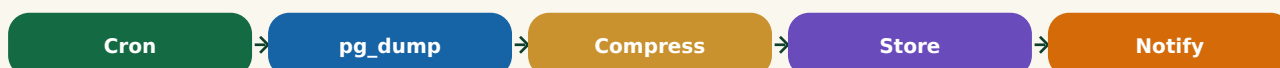
Ưu điểm	Nhược điểm
Giữ content hub nhất quán, giảm lỗi SEO kỹ thuật cơ bản.	Không thay người biên tập; description và category vẫn cần duyệt thủ công.

Quy trình 4: Backup dữ liệu Supabase/PostgreSQL ra file định kỳ

Vấn đề giải quyết

Dự án nhỏ thường quên backup. Khi lỡ xóa bảng, deploy sai migration hoặc bị lỗi dữ liệu, không có đường quay lại.

Luồng xử lý



Các bước triển khai

1. Tạo script backup dùng biến môi trường DATABASE_URL.
2. Nén file và đặt tên theo ngày.
3. Lưu vào folder backup hoặc storage riêng.
4. Thông báo kết quả.
5. Định kỳ thử restore vào database test.

Code mẫu / khung triển khai

```
const { execFile } = require('child_process');
const path = require('path');
const fs = require('fs');

const date = new Date().toISOString().slice(0, 10);
const out = path.join('backups', `db-${date}.dump`);
fs.mkdirSync('backups', { recursive: true });

execFile('pg_dump', [process.env.DATABASE_URL, '-Fc', '-f', out], (err) => {
  if (err) {
    console.error('Backup failed', err);
    process.exit(1);
  }
  console.log(`Backup saved: ${out}`);
});
```

Ưu và nhược điểm

Ưu điểm	Nhược điểm
Tăng an toàn vận hành, đặc biệt khi website có admin/CMS.	Cần bảo mật file backup vì nó có thể chứa dữ liệu người dùng.

Quy trình 5: Báo cáo KPI tuần tự động từ dữ liệu website/resource

Vấn đề giải quyết

Bạn có tài nguyên miễn phí nhưng không biết tài nguyên nào được tải nhiều, bài nào kéo lead tốt, hoặc tuần này có gì tăng/giảm.

Luồng xử lý



Các bước triển khai

1. Lấy dữ liệu leads/downloads/page views.
2. Tính tổng theo tuần.
3. Tìm top tài nguyên và nguồn chuyển đổi.
4. Tạo báo cáo Markdown.
5. Gửi email hoặc lưu vào Notion.

Code mẫu / khung triển khai

```
function summarizeWeekly(leads) {  
  const byResource = {};  
  for (const lead of leads) {  
    byResource[lead.resourceId] = (byResource[lead.resourceId] || 0) + 1;  
  }  
  return Object.entries(byResource)  
    .sort((a, b) => b[1] - a[1])  
    .map(([resource, count]) => `${resource}: ${count} lượt tải`)  
    .join('');  
}
```

```
async function weeklyReport() {  
  const leads = await prisma.lead.findMany({  
    where: { createdAt: { gte: getStartOfWeek() } }  
  });  
  const report = `# Báo cáo tài nguyên tuần này`  
}
```

```
${summarizeWeekly(leads)}`;  
await fs.promises.writeFile('weekly-report.md', report);  
}
```

Ưu và nhược điểm

Ưu điểm	Nhược điểm
Giúp ra quyết định content/resource bằng dữ liệu thay vì cảm tính.	Số liệu nhỏ dễ gây ảo tưởng. Cần nhìn xu hướng nhiều tuần, không kết luận quá sớm.

4. Checklist trước khi chạy automation thật

01	[]	Đã lưu token/API key trong .env, không hardcode vào source.
02	[]	Đã có validate input và xử lý dữ liệu thiếu.
03	[]	Đã có log thành công/thất bại.
04	[]	Đã chạy thử trên dữ liệu mẫu.
05	[]	Đã có cách rollback hoặc backup nếu script ghi sai dữ liệu.
06	[]	Đã giới hạn quyền API ở mức tối thiểu cần dùng.
07	[]	Đã ghi lại lịch chạy và người chịu trách nhiệm kiểm tra.